Top-K Heavy Hitters

count in fixed memory, merge, compare with normal

Watch the 4-min video

In one line: to find the K most frequent items in a huge stream, count approximately in **fixed memory** (Count-Min Sketch + min-heap, or the Space-Saving algorithm), count in **time buckets** so results reflect the recent window, score against a **baseline** so "trending" means unusual, and **merge local top lists** from partitioned workers into a global top K.

1 Trending hashtags, right now

Thousands of posts arrive every second containing hundreds of thousands of different hashtags, most seen once or twice. The product wants the top 10 of the last hour, updated every few seconds, and it must survive the spike when something big happens. Keeping an exact counter for every hashtag ever seen, for every minute, is wasteful; most of that memory goes to the long tail that will never trend. This is the **top-K** or **heavy hitters** problem.

2 The exact way: batch

For yesterday's top hashtags, a MapReduce or Spark job is exact and simple: map each post to its hashtags, reduce by hashtag to sum counts, then each reducer keeps its **local top K** and a final step merges them. Because the shuffle sends every occurrence of a hashtag to one reducer, each count is complete and the merged top K is exact. This is the right answer for reports. It is not the answer for "last hour, every 10 seconds".

3 Streaming in fixed memory

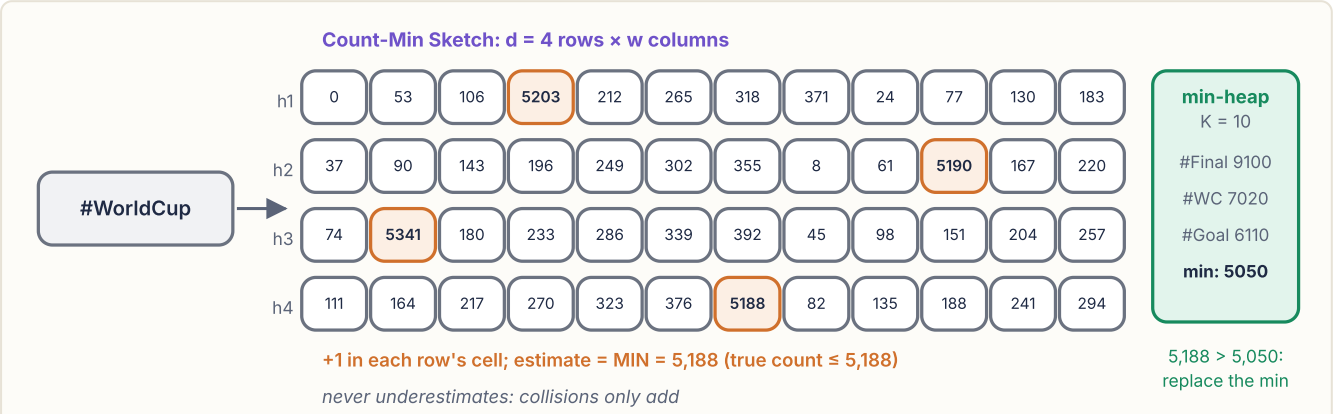


Figure 1. Each occurrence increments one counter per row; the estimate is the minimum. A size-K min-heap keeps the current leaders.

Count-Min Sketch + heap. The sketch holds approximate counts for *all* items in d × w counters (lesson 39). For each arrival: increment the sketch, read the estimate, and if it beats the heap's minimum, replace the minimum (or update the item's entry if it is already in the heap, using a side map). Memory is fixed, O(d·w + K), however many distinct hashtags appear. The error is one-sided: a rare tag that collides with popular ones may look bigger than it is, never smaller.

Space-Saving (Metwally et al., 2005) is often better in practice: keep exactly m counters. A tracked item increments its own counter; an untracked item, when all counters are in use, **evicts the item with the smallest count** and inherits that count + 1. Each count overestimates by at most the evicted minimum, and any item occurring more than N/m times is guaranteed to be tracked.

Arrival	Counters (m = 3)	What happened
a a b c	a:2 · b:1 · c:1	three items fit
d	a:2 · **d:2** · c:1	d evicts b (min 1) → 1 + 1 = 2 (over by ≤ 1)
a	a:3 · d:2 · c:1	tracked item: plain increment
e	a:3 · d:2 · **e:2**	e evicts c (min 1); a stays on top

4 Recent, and unusual

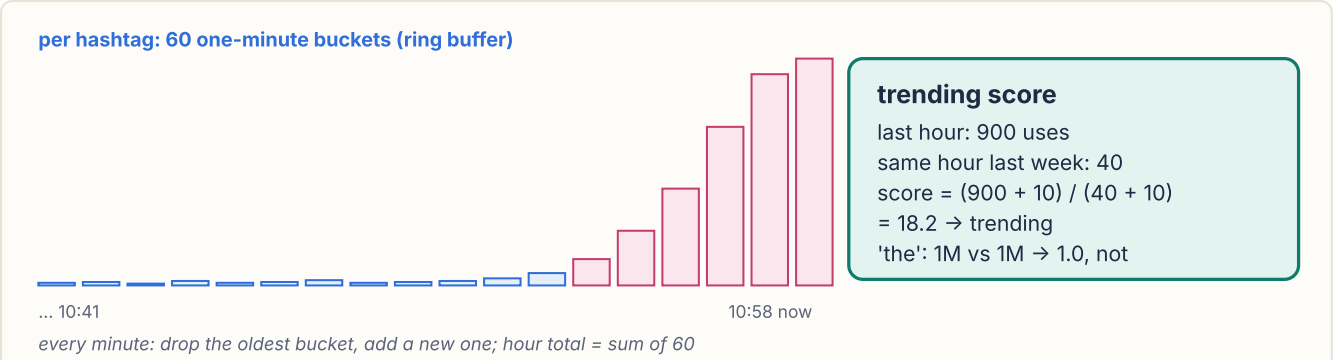


Figure 2. Bucketed counts give a sliding hour; comparing with a baseline separates a spike from a word that is always common.

Windows. Keep 60 one-minute buckets per worker (each a count map or its own sketch). The hour's count is the sum of 60; every minute the oldest bucket is dropped, so items fall out cleanly. An alternative is **exponential decay**: `score = score · e^{−λΔt} + 1` on each arrival, with λ set from a half-life (say 30 minutes); old activity fades gradually and only one number per item is stored.

Popular is not trending. "the" or #love is always near the top. Trending compares the current rate with what is normal for that item: the same hour last week, or a long-term moving average. A smoothed ratio `(current + α) / (baseline + α)` stops tiny baselines from producing huge scores; statistical versions use a z-score. Add guards: a minimum absolute count, and a minimum number of **distinct users** per hashtag (an HLL per tag, lesson 38), so one bot posting 10,000 times can't trend.

5 Distributing the work

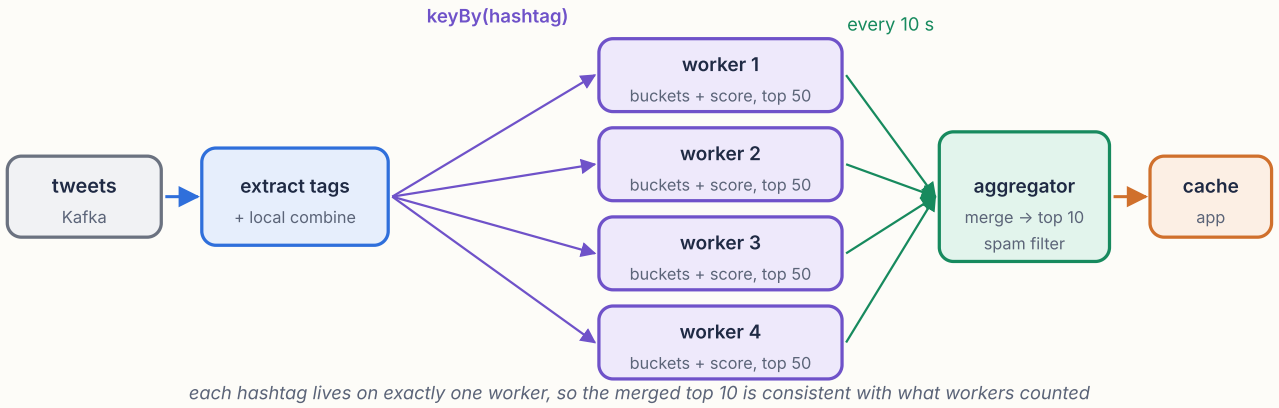


Figure 3. Partition by hashtag, pre-combine upstream, and merge small local top lists centrally every few seconds.

Partition the stream by hashtag (`keyBy`), so every occurrence of a tag reaches the same worker and its count there is complete. Each worker keeps buckets, scores, and a local top list; every 10 seconds it sends its top 50 to an aggregator that merges them into the global top 10. Two subtleties:

- **If partitioning is not by item** (e.g. posts spread round-robin), each worker sees only part of each item's count. An item moderately popular everywhere may miss every local top 10 yet be first globally. Send more candidates than K (top 50 for a top 10) and, for exactness, re-query the candidates' counts from all workers.
- **Hot keys:** keying by hashtag puts a viral tag's entire load on one worker. Pre-aggregate upstream (count per tag per second before the shuffle), which turns a million events into one message per producer per second.

WORKED EXAMPLE: TRENDING HASHTAGS IN THE LAST HOUR

Posts flow through Kafka to a Flink job that extracts hashtags, pre-sums them per second and keys by hashtag across 32 workers. Each worker keeps 60 one-minute count buckets per tag it owns, plus last week's hourly baseline loaded from a table, and an HLL of distinct authors. #WorldCupFinal has 900 uses this hour from 640 authors against a baseline of 40: score (900 + 10) / (40 + 10) ≈ 18. Every 10 seconds each worker sends its top 50 by score to an aggregator, which merges 1,600 candidates, drops blocked terms and spam, and writes the top 10 to a cache key that the app reads; the whole list is at most about 10 seconds old.

TRADE-OFFS

- Exact counting is simple but memory grows with distinct items; sketches and Space-Saving fix memory at the cost of bounded error.
- More candidates per worker (50 vs 10): more accurate global top K, more merge traffic.
- Shorter windows or faster decay react quickly but are noisier and easier to game.
- Baseline scoring finds novelty but needs history; brand-new tags need a sensible default baseline.

COMMON MISTAKES

- Ranking by raw count, so the same evergreen words always "trend".
- Merging local top-K lists from workers that each saw only part of every item's count, and calling it exact.
- No defence against one account or a bot ring pushing a tag.
- Keying by hashtag without pre-aggregation, so a viral tag overloads one worker.

6 Key terms

Heavy hitters: items whose frequency exceeds a threshold of the stream

Top-K: the K most frequent (or highest-scoring) items

Count-Min Sketch + heap: approximate counts for all, exact list of leaders

Space-Saving: m counters; new items evict the minimum and inherit it

Time buckets: per-minute counts summed into a sliding window

Baseline: normal rate for an item, used to score trending

7 Interview questions

Q1. Design a top-K trending system. BASIC

Partition the stream; workers count in time buckets (Count-Min Sketch + heap if the item space is huge), score against a baseline, and merge local top lists into a global top K every few seconds.

Q2. Exact or approximate? BASIC

Exact for batch reports; approximate for real-time streams, where fixed memory matters more than perfect counts.

Q3. The word 'the' is always trending. Fix it. SCENARIO

Score by the ratio of the current rate to the item's normal rate (same hour last week or moving average), with smoothing and minimum counts.

Q4. A bot ring makes a hashtag trend. What do you add? SCENARIO

Count distinct authors per tag (HLL) with a minimum, weight by account quality, and filter spam and blocked terms in the aggregator.

Q5. Why might merging each worker's top 10 miss the true global #1? TRICKY

If workers don't own whole items, an item can be 11th on every worker but first in total; send more candidates or partition by item.

Q6. What does Space-Saving guarantee? TRICKY

With m counters, every item with frequency above N/m is tracked, and each reported count overestimates by at most the minimum counter value.

Q7. Sliding buckets or exponential decay? TRICKY

Buckets give an exact, explainable 'last hour' at 60× the counters; decay needs one number per item and fades smoothly, but has no hard window.

Next: SD-R-001 · Tenant Isolation backendroadmap.com